Support Ticket Classifier Documentation

Overview

The Support Ticket Classifier is an AI-powered ticket triage system that processes customer support messages and returns a structured classification response for downstream support workflows. The application classifies each ticket into an issue category, assigns the appropriate team, predicts priority, analyzes user sentiment, computes a confidence score, and determines whether human review is required.

The system is built with FastAPI, LangGraph, and OpenRouter-backed LLM inference. It also includes PII redaction, prompt injection detection, response validation, prompt versioning, fallback retry logic, and cost tracking as part of the production workflow.

Project Purpose

The classifier automates the initial support triage process by transforming unstructured customer messages into consistent machine-readable outputs. This helps reduce manual effort, improve routing speed, and ensure support requests are assigned to the right operational teams.

Core Features

- Issue category classification.
- Team assignment.
- Priority prediction.
- User sentiment analysis.
- Confidence scoring.
- Human review flagging.
- PII detection and redaction.
- Prompt injection detection and blocking.
- Prompt versioning.
- Cost and token tracking.
- Fallback retry logic for invalid or failed model outputs.

Project Structure

```
support-ticket-classifier/
├── main.py
├── graph.py
├── schema.py
├── productionmodules/
│   ├── __init__.py
│   ├── structuredoutput.py
│   ├── validateresponse.py
│   ├── nondeterminism.py
│   ├── piiredaction.py
│   ├── promptinjection.py
│   ├── promptversioning.py
│   ├── costcalculator.py
│   └── fallbackretry.py
├── demoui/
│   └── index.html
├── tests/
│   └── testclassifier.py
├── .env
├── .env.example
├── requirements.txt
└── documentation.md
```

This structure organizes the API layer, graph orchestration, schema definitions, production support modules, frontend demo UI, and automated tests into a single project layout.

Main Components

main.py

`main.py` contains the FastAPI application and HTTP entry points. It validates incoming requests, invokes the pipeline, and formats the final JSON response returned to the client.

graph.py

`graph.py` orchestrates the support-ticket workflow through LangGraph. The pipeline connects redaction, guard checks, classification, validation, fallback handling, and cost logging into a deterministic execution flow.

schema.py

`schema.py` defines the Pydantic models and enums used across the application. These schemas enforce structured output and consistent API responses.

productionmodules

The `productionm odules` package contains the main operational modules used by the graph pipeline.

- `structuredoutput. py`: Handles structured LLM classification calls.
- `validateresponse. py`: Validates classification output against schema and business rules.
- `nondeterm inism . py`: Supports deterministic and consistency-related checks.
- `piiredaction. py`: Detects and redacts email, phone, and credit card data.
- `prom ptinjection. py`: Checks tickets for prompt injection attempts.
- `prom ptversioning. py`: Stores and serves versioned prompt templates.
- `costcalculator. py`: Tracks token usage and total estimated cost.
- `fallbackretry. py`: Retries or falls back when classification output is invalid or uncertain.

Classification Schema

The classifier returns a strongly structured ticket object with validated fields. The project uses enums for issue category, team ownership, priority, and sentiment labels.

Issue Categories

- `order_issue`.
- `paym ent_issue`.
- `delivery_issue`.
- `product_issue`.
- `account_issue`.
- `refund_request`.
- `other`.

Team Owners

- `fulfillm ent_team` .
- `paym ents_team` .
- `logistics_team` .
- `custom er_support`.
- `tech_team` .

Priority Levels

- `low`.
- `medium`.
- `high`.
- `critical`.

Sentiment Labels

- `positive`.
- `neutral`.
- `negative`.
- `angry`.

Response Fields

The classification response includes the following fields.

- `issue_category`.
- `assigned_team`.
- `priority`.
- `user_sentiment`.
- `confidence_score`.
- `reasoning`.
- `requires_human_review`.
- `pii_detected`.
- `prompt_version`.
- `injection_blocked`.
- `cost_info`.

API Endpoints

The application runs as a FastAPI service on port 8000 in local development. Interactive API documentation is available through `/docs` and `/redoc`.

GET /health

Returns a simple status response indicating that the API is running.

GET /prompts

Returns prompt version metadata and active prompt configuration information.

POST /classify

Processes a support ticket and returns the structured classification result. This endpoint is used by the demo UI and can also be tested directly with tools such as cURL or Postman.

Request Body

```
{
  "ticket_text": "I was charged twice for order 9981. Please refund immediately!",
  "channel": "web_form"
}
```

Supported channel values include `web_form` and `email`.

Response Body

```
{
  "issue_category": "payment_issue",
  "assigned_team": "payments_team",
  "priority": "high",
  "user_sentiment": "angry",
  "confidence_score": 0.97,
  "reasoning": "Customer explicitly reports a duplicate charge and requests a refun
  "requires_human_review": false,
  "pii_detected": false,
  "prompt_version": "v2",
  "injection_blocked": false,
  "cost_info": {
    "model": "openrouter/model-name",
    "input_tokens": 134,
    "output_tokens": 68,
    "total_cost_usd": 0.000061
  }
}
```

Workflow Pipeline

Each support ticket passes through a staged processing pipeline managed by LangGraph.

1. The client sends a `POST /classify` request with ticket text and channel.
2. The API validates the request and forwards it to the pipeline.
3. `piiredactnode` redacts sensitive information such as email addresses, phone numbers, and credit card numbers.

4. `injectioncheck node` evaluates the raw ticket for prompt injection or unsafe instructions.
5. If the input is safe, `classifynode` sends the redacted ticket to the structured classification module.
ú. `validatenode` checks the returned output against schema and business rules.
7. If validation fails, `fallbacknode` retries classification with a simpler prompt or returns a safe fallback response.
8. `costlognode` calculates token usage and estimated request cost.
9. The API returns the final validated response to the caller.

If prompt injection is detected, the pipeline blocks normal classification and returns a guarded fallback output instead.

PII Redaction

The redaction module scans ticket text for sensitive personal information before LLM classification. The template identifies support for email addresses, phone numbers, and credit card data.

Example replacement markers include:

- `EMAIL_REDACTED`.
- `PHONE_REDACTED`.
- `CREDIT_CARD_REDACTED`.

The redaction result includes the cleaned text, detected entity types, and a `pii_detected` flag.

Prompt Injection Detection

The guard module evaluates incoming tickets for prompt injection patterns such as role override attempts or malicious instruction hijacking. Unsafe inputs are blocked before the main classification step and returned through a safe guarded path.

This protects the classifier from adversarial user input and prevents unsafe prompt manipulation from reaching the classification model.

Prompt Versioning

The application supports prompt versioning to manage and test different classifier instructions. The template includes at least two prompt variants.

- `v1`: Basic classification prompt.
- `v2`: Enhanced prompt with stronger reasoning and human review guidance.

The active prompt version is selected through environment configuration and is also returned in the final API response.

Validation Rules

The response validation layer checks that the LLM output matches the expected schema and satisfies business constraints. Confidence scores must remain within valid numeric ranges, and high-risk classifications can trigger required human review.

The template explicitly notes that critical-priority tickets should require human review. This validation stage prevents invalid or unsafe model responses from being returned directly to the client.

Fallback and Retry

If the primary structured classification output is malformed, inconsistent, or fails validation, the system retries classification using a simplified fallback prompt. If repeated attempts fail, the API returns a safe fallback classification with low confidence and `requires_human_review` set to `true`.

This ensures the application always returns a predictable response structure even when the model output is unreliable.

Cost Tracking

The system tracks token usage and estimated request cost for each classification request. The `cost_info` field includes the active model name, input token count, output token count, and total estimated cost.

This information is also surfaced in the demo UI status strip so that users can see runtime usage details after classification.

Frontend Demo UI

The project includes a single-file browser demo interface that sends requests to `http://localhost:8000/classify`. The UI contains sample ticket buttons, a textarea for ticket input, a channel selector, a classify button, a result panel, and a bottom status strip for tokens, cost, PII status, and prompt version.

The result panel displays issue category, assigned team, priority, sentiment, confidence score, reasoning, and any human review or injection-blocked indicators returned by the API.

Environment Configuration

The application uses environment-based configuration for runtime behavior.

```
OPENROUTER_API_KEY=your-openrouter-api-key-here
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
PROMPT_VERSION=v2
DEFAULT_MODEL=openrouter/model-name
LOG_COSTS=true
```

The API key authenticates requests, the base URL points the client to OpenRouter, the prompt version selects the active prompt template, the default model controls the classification model, and `LOG_COSTS` enables request cost logging.

Installation and Setup

1. Move into the project directory

```
cd support-ticket-classifier
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Copy the environment template

```
cp .env.example .env
```

4. Configure `.env`

Add the OpenRouter API key, prompt version, and default model in the environment file.

5. Start the server

```
python3 -m uvicorn main:app --reload --port 8000
```

6. Open the application

- API root: `http://localhost:8000`.
- Swagger docs: `http://localhost:8000/docs`.
- ReDoc: `http://localhost:8000/redoc`.

Example cURL Request

```
curl -X POST http://localhost:8000/classify \
  -H "Content-Type: application/json" \
  -d '{
    "ticket_text": "I was charged twice for order 9981. Please refund immediately!"
    "channel": "web_form"
  }'
```

Testing

The template includes a pytest suite for functional and safety checks. Mentioned tests include normal delivery classification, PII redaction before LLM calls, prompt injection blocking, ambiguous ticket handling, and fallback behavior after bad output.

Run the tests with:

```
pytest tests/testclassifier.py -v
```

Summary

The Support Ticket Classifier combines FastAPI, LangGraph orchestration, OpenRouter-based LLM classification, PII redaction, prompt injection protection, validation, fallback handling, prompt versioning, and cost tracking into a single production-style support triage system. It is designed to classify support tickets safely, consistently, and in a structured format suitable for frontend interfaces and operational workflows.